

O analiză experimentală a algoritmilor de sortare uzuali. Când Sleep Sort câștigă.

Cristian Meriacri^{*,a,1}

^a Universitatea de Vest din Timișoara, Facultatea de Informatică, Str. Pârvan 4, Timișoara 300223, Romania - cristian.meriacrioo@e-uvt.ro

Acest articol a fost redactat în perioada Februarie-Mai 2026.

Abstract

Această lucrare prezintă o analiză experimentală a performanței algoritmilor de sortare, comparând soluții clasice precum Bubble Sort, Insertion Sort, Quick Sort, Merge Sort și implementarea de referință din biblioteca standard C++, `std::sort`. Un element central și neconvențional al studiului este includerea algoritmului Sleep Sort, testat pentru a identifica limitele practice ale eficienței bazate pe multi-threading.

Metodologia a presupus implementarea în C++ folosind compilatorul GCC și testarea pe diverse seturi de date, cu dimensiuni cuprinse între 10 și 2.000.000 de elemente și cinci distribuții de valori diferite. Principalul parametru monitorizat a fost timpul de execuție, măsurat în timpul real de pe ceasul de perete folosind librăria standard inclusă `chrono`, pentru a evalua scalabilitatea și performanța fiecărui algoritm.

Rezultatele obținute au evidențiat diferențe semnificative în performanță, cu algoritmi precum Quick Sort și Merge Sort demonstrând o eficiență superioară în comparație cu soluțiile mai simple. Totuși, deși majoritatea rezultatelor au fost pe măsura așteptărilor, bazat pe ce știm despre algoritmii de sortare, Sleep Sort a prezentat un comportament neașteptat, depășind performanța algoritmului `std::sort` din librăria C++ în anumite condiții.

Lucrarea demonstrează astfel că eficiența unui algoritm este strâns legată de natura distribuției datelor și de dimensiunea acestora, evidențiind importanța alegerii algoritmului potrivit pentru fiecare situație specifică.

Keywords: Algoritmi, Sortare, Bubble Sort, Insertion Sort, Quick Sort, Merge Sort, Sleep Sort, `std::sort`, C++, Performanță, multi-threading, `chrono`

CONTENTS

1	Introducere	2	9	Equations	6
1.1	Declarație de originalitate	2	10	Codes	7
2	Date de Intrare	2	10.1	Coding with Minted*	7
2.1	Generalități	2	10.2	Coding with Listings	7
2.2	Tipuri de date	2	10.3	Inline Code*	7
	Aleatoriu • Sortat Crescător • Sortat Descrescător • Aproape Sortat • Plat		Numberless section	7	
3	Algoritmi de sortare	2	11	Bibliography	7
3.1	Bubble Sort	2	FAQ	7	
3.2	Insertion Sort	2	How do I manage my references?	7	
3.3	Merge Sort	3	What should I do with the example files?	8	
3.4	Quick Sort	3	How do I place equations easily?	8	
3.5	<code>std::sort</code>	3	How do I change the paper size?	8	
3.6	Sleep Sort	4	Contact me	8	
	Determinarea timpului de <i>somn</i> • Problema elementelor duplicate		Github Repository	8	
	• Complexitatea algoritmului și influența sistemului de operare și hardware-ului • Comparația cu Counting și Radix Sort		Supporting	8	
3.7	Ipoteze și așteptări	5	Any contributions are welcome!	8	
4	Cross-Reference Commands*	5			
5	Figures	5			
6	Tables	6			
7	Document Data	6			
7.1	Front-Matter Footer Block*	6			
7.2	Headers and Footers*	6			
	Headers • Footers				
8	Rho Custom Packages	6			
8.1	Rhobabel.sty	6			
8.2	Rhoenvs.sty	6			

1. INTRODUCERE

Performanța unui program depinde adesea mai mult de alegerea algoritmului decât de orice altă optimizare. În contextul sortării, una dintre operațiile cele mai frecvente în software, această alegere poate face diferența pe date reale, care rareori sunt perfect distribuite.

Această lucrare prezintă o analiză experimentală a șase algoritmi de sortare pe distribuții diverse, cu accent pe comportamentul lor în scenarii nefavorabile. Un element distinctiv este includerea Sleep Sort, un algoritm neconvențional bazat pe concurență, pentru a investiga dacă poate fi competitiv în condiții specifice.

Lucrarea este organizată astfel: Secțiunea ?? descrie algoritmi, Secțiunea ?? prezintă configurația experimentală, Secțiunea ?? analizează rezultatele, iar Secțiunea ?? sintetizează concluziile.

1.1. Declarație de originalitate

Implementarea Sleep Sort propusă este propria mea contribuție: am observat că varianta clasică creează câte un thread per element, inefficient pe date cu duplicate, și am rezolvat asta grupând elementele identice printr-un `std::map` pe același fir de execuție.

2. DATE DE INTRARE

2.1. Generalități

Această secțiune oferă o prezentare generală a datelor folosite în această lucrare, evidențiind principiile de funcționare și complexitatea lor teoretică. Algoritmii de sortare (mai puțin Sleep Sort) pot sorta orice tip de date, atâta timp cât există o relație de ordine definită între ele.

Pe parcursul acestei lucrări, toate datele de intrare sunt de tip integer.

2.2. Tipuri de date

Fie $n \in \mathbb{N}$ dimensiunea șirului și $A = (a_1, a_2, \dots, a_n)$ un șir de numere întregi, $a_i \in \mathbb{Z}$. Algoritmii au fost testați pe cinci distribuții de date, alese pentru a acoperi scenarii reprezentative din practică.

2.2.1. Aleatoriu

A este generat aleatoriu, unde fiecare $a_i \sim \mathcal{U}(1, 10^6)$, independent și uniform distribuit. Aceasta reprezintă cazul general, fără nicio structură prealabilă în date.

2.2.2. Sortat Crescător

A este un șir sortat crescător, unde $a_i \leq a_{i+1}$ pentru orice $i = 1, \dots, n-1$. Reprezintă cel mai favorabil scenariu pentru algoritmii care exploatează ordinea existentă.

2.2.3. Sortat Descrescător

A este un șir sortat descrescător, unde $a_i \geq a_{i+1}$ pentru orice $i = 1, \dots, n-1$. Reprezintă cazul defavorabil pentru majoritatea algoritmilor bazați pe comparații.

2.2.4. Aproape Sortat

A este inițial sortat crescător, după care $\lfloor 0.01 \cdot n \rfloor$ perechi de elemente sunt interschimbate aleatoriu. Astfel, aproximativ 1% din elemente sunt în afara poziției corecte, simulând date reale care au suferit modificări minore după o sortare anterioară.

2.2.5. Plat

A este generat aleatoriu, unde $a_i \in \{v_1, v_2, v_3, v_4, v_5\}$, un set fix de exact 5 valori distincte. Această distribuție testează comportamentul algoritmilor în prezența unui număr foarte mare de duplicate.

3. ALGORITMI DE SORTARE

Această secțiune oferă o prezentare generală a algoritmilor de sortare analizați în această lucrare, evidențiind principiile de funcționare și complexitatea lor teoretică.

3.1. Bubble Sort

Bubble Sort este cel mai simplu algoritm de sortare, care funcționează prin compararea elementelor adiacente și interschimbarea lor dacă sunt în ordine greșită. Acest proces se repetă până când întregul șir este sortat. Deși este un algoritm foarte inefficient, Bubble Sort este adesea folosit în scop educațional pentru a ilustra conceptele de bază ale sortării.

Origine și popularitate

Originea acestui algoritm este incertă [6], fiind menționat sub diferite denumiri în literatura de specialitate ca "Sorting by Exchange", "Sinking Sort", "Jump Down", "Sink Down" sau "Exchange Sort".

Knuth l-a inclus în *The Art of Computer Programming* [4], contribuind la consolidarea lui în literatura de specialitate, deși l-a criticat explicit.

Complexitatea [4] a acestui algoritm este $O(n^2)$ în varianta de bază, fără optimizări, și $O(n)$ în cel mai favorabil caz (când șirul este deja sortat) dacă este folosită o implementare optimizată. În medie, complexitatea este tot $O(n^2)$, ceea ce îl face inefficient pentru șiruri mari.

Totodată, Bubble sort e mai încet decât alți algoritmi de sortare cu o complexitate $O(n^2)$ precum Insertion Sort sau Selection Sort (cu unele excepții evidențiate mai târziu), deoarece necesită mai multe interschimbări [6] pentru a sorta șirul.

Complexitatea de spațiu a acestui algoritm este $O(1)$, deoarece sortarea se face in-place (fără a alocă structuri de date auxiliare proporționale cu n), fără a necesita spațiu suplimentar semnificativ.

```
template <typename T>
void bubbleSort(std::vector<T>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; i++) {
        bool swapped = false;
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                std::swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }
        if (!swapped) break;
    }
}
```

Code 1. Implementarea Bubble Sort în C++.

După cum se poate observa în codul sursă 1, algoritmul parcurge șirul de la început până la sfârșit, comparând elementele adiacente și interschimbându-le dacă sunt în ordine greșită. Un flag numit `swapped` este folosit pentru a verifica dacă s-au făcut interschimbări în timpul unei treceri. Dacă nu s-a făcut nicio interschimbare, înseamnă că șirul este deja sortat și algoritmul se oprește, optimizând astfel cazul favorabil.

3.2. Insertion Sort

Insertion Sort funcționează prin construirea treptată a șirului sortat, inserând fiecare element în poziția corectă în raport cu cele deja sortate. Prima mențiune documentată îi aparține lui John Mauchly în 1946, deși algoritmul era probabil folosit informal înainte [4].

Algoritmul este eficient pentru șiruri mici sau aproape sortate, având o complexitate de $O(n)$ în cel mai favorabil caz și $O(n^2)$ în cazul mediu și defavorabil [4]. Complexitatea de spațiu este $O(1)$, deoarece sortarea se face in-place.

Knuth menționează că poziția de inserare poate fi găsită prin căutare binară [4], reducând numărul de comparații la $O(\log n)$, fără a îmbunătăți însă complexitatea totală. O variantă mai agresivă, Gap Insertion Sort, folosește goluri între elemente pentru a reduce și mutările, obținând $O(n \log n)$ în majoritatea cazurilor [7].

```
template <typename T>
void insertionSort(std::vector<T>& arr) {
    int n = arr.size();
    for (int i = 1; i < n; i++) {
        T key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}
```

Code 2. Implementarea Insertion Sort în C++.

Implementarea Insertion Sort prezentată în codul sursă 2 este o versiune clasică, fără optimizări ca căutarea binară pentru poziția de inserare menționată anterior.

3.3. Merge Sort

Merge Sort este un algoritm de sortare bazat pe paradigma divide et impera, care împarte șirul în subsiruri mai mici, le sortează recursiv și apoi le combină pentru a obține șirul sortat final. A fost inventat de John von Neumann în 1945 [4]. Eficiența sa este de $O(n \log n)$ în toate cazurile, fără surprize sau cazuri excepționale, ceea ce îl face potrivit pentru șiruri mari.

Un dezavantaj al acestui algoritm este complexitatea de spațiu, care este $O(n)$, deoarece necesită un spațiu auxiliar pentru a stoca subsirurile în timpul procesului de combinare. Totodată, natura recursivă a algoritmului necesită un spațiu suplimentar pentru stiva de apeluri, ceea ce îl face inefficient pentru șiruri foarte mici sau pentru sisteme cu resurse limitate.

```
template <typename T>
void merge(std::vector<T>& arr, int left, int mid, int right)
{
    int n1 = mid - left + 1;
    int n2 = right - mid;
    std::vector<T> L(n1), R(n2);

    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) { arr[k] = L[i]; i++; }
        else { arr[k] = R[j]; j++; }
        k++;
    }
    while (i < n1) { arr[k] = L[i]; i++; k++; }
    while (j < n2) { arr[k] = R[j]; j++; k++; }
}

template <typename T>
void mergeSort(std::vector<T>& arr, int left, int right) {
    if (left >= right) return;
    int mid = left + (right - left) / 2;
    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);
    merge(arr, left, mid, right);
}
```

Code 3. Implementarea Merge Sort în C++.

Implementarea prezentată în codul sursă 3 folosește două funcții: `merge`, care combină două subsiruri sortate într-unul singur, și `mergeSort`, care realizează divizarea recursivă a șirului și apelează funcția de combinare.

3.4. Quick Sort

Quick Sort este un algoritm de sortare eficient, bazat pe paradigma divide et impera, care a fost inventat de Tony Hoare în 1959 și publicat în 1961 [1]. Algoritmul funcționează prin alegerea unui element pivot din șir și împărțirea celorlalte elemente în două subsiruri: cele mai mici decât pivotul și cele mai mari decât pivotul. Acest proces se repetă recursiv pentru fiecare subsir până când întregul șir este sortat.

Quick Sort, în ciuda numelui său, nu garantează performanță superioară în toate cazurile [1, 4].¹ O problemă cunoscută a acestui algoritm este că performanța sa poate degrada la $O(n^2)$ în cazul în care pivotul ales este întotdeauna cel mai mic sau cel mai mare element, cum ar fi atunci când șirul este deja sortat sau sortat descrescător. Cu toate acestea, cu o alegere bună a pivotului (de exemplu, folosind metoda medianei), complexitatea medie și cea mai bună rămân $O(n \log n)$ [4]. O concepție comună este că Quick Sort este întotdeauna mai rapid decât Merge Sort sau Bubble Sort, dar acest lucru nu este întotdeauna adevărat, deoarece Quick Sort are nevoie de mai mult overhead pentru gestionarea recursiei și poate fi inefficient pe șiruri mici sau aproape sortate, unde Insertion Sort sau Bubble Sort pot performa mai bine [4]. Aceeași problemă o suferă și Merge Sort. Complexitatea de spațiu a Quick Sort este $O(\log n)$ în medie, datorită stivei de apeluri recursive, dar poate ajunge la $O(n)$ în cel mai defavorabil caz.

O optimizare separată, numită Algoritmul Flagului Olandez [2], propusă de Dijkstra, modifică partiționarea pentru a gestiona eficient elementele duplicate — în loc de două zone (mai mic și mai mare decât pivotul), șirul este împărțit în trei: mai mic, egal și mai mare. Aceasta reduce semnificativ numărul de operații pe distribuții cu multe duplicate, cum ar fi distribuția plată din această lucrare.

```
template <typename T>
int partition(std::vector<T>& arr, int low, int high) {
    T pivot = arr[high];
    int i = (low - 1);
    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i++;
            std::swap(arr[i], arr[j]);
        }
    }
    std::swap(arr[i + 1], arr[high]);
    return (i + 1);
}

template <typename T>
void quickSort(std::vector<T>& arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

Code 4. Implementarea Quick Sort în C++.

Pentru a ilustra consecințele alegerii pivotului, implementarea prezentată în codul sursă 4 folosește ultimul element ca pivot, ceea ce poate duce la performanță slabă pe șiruri deja sortate sau sortate descrescător. Pentru a evita acest lucru, se pot implementa strategii de alegere a pivotului, cum ar fi alegerea unui element aleatoriu sau folosirea medianei a trei elemente.

3.5. std::sort

`std::sort` este o funcție de sortare din biblioteca standard C++ care oferă o implementare optimizată, adaptată la dimensiunea și structura datelor. De obicei, `std::sort` utilizează o combinație de Quick Sort, Heap Sort și Insertion Sort, denumită Introsort [3, 15].

Implementarea specifică poate varia între diferite compilatoare și platforme. În compilatorul GCC, `std::sort` folosește o variantă a

¹ Observație atribuită cursului de Algoritmi și Structuri de Date, Universitatea de Vest din Timișoara, de Prof. Dr. D. Zaharie.

Introsort care începe cu Quick Sort și comută la Heap Sort dacă adâncimea recursivă depășește un prag specificat, prevenind astfel degradarea la $O(n^2)$ pe șiruri sortate sau aproape sortate. Pentru subșiruri foarte mici, algoritmul folosește Insertion Sort, mai eficient în acest context.

Complexitatea `std::sort` este $O(n \log n)$ în toate cazurile [3], inclusiv cel defavorabil. Complexitatea de spațiu este $O(\log n)$, datorită stivei de recursivitate. Algoritmul nu este stabil — pentru stabilitate, biblioteca standard oferă `std::stable_sort`.

Codul sursă al `std::sort` nu este unic, fiind adaptat diferitelor platforme și compilatoare. O implementare de referință a Introsort poate fi găsită în literatura de specialitate [3], iar implementarea din GCC este disponibilă în fișierul `bits/stl_algo.h` [16].

```
template<typename _RandomAccessIterator, typename _Compare>
inline void
__sort(_RandomAccessIterator __first, _RandomAccessIterator
    __last,
    _Compare __comp)
{
    if (__first != __last)
    {
        std::__introsort_loop(__first, __last,
            std::__lg(__last - __first) * 2,
            __comp);
        std::__final_insertion_sort(__first, __last, __comp);
    }
}
```

Code 5. Funcția "caller" pentru `std::sort` în `stl_algo.h`.

În această lucrare, `std::sort` are rolul de referință pentru evaluarea algoritmilor implementați manual, oferind un punct de reper reprezentând o soluție optimizată și bine testată în scenarii diverse de date.

O misconcepție foarte răspândită printre fanii și puriștii limbajului C++ este că `std::sort` este cel mai rapid algoritm de sortare posibil, dar acest lucru nu este întotdeauna adevărat.

Deși `std::sort` este extrem de optimizat și oferă performanță excelentă în majoritatea cazurilor, există algoritmi mai eficienți pentru date reale.

Un exemplu este Timsort [5], un algoritm hibrid care combină Merge Sort și Insertion Sort, optimizat pentru date care conțin secvențe deja sortate. Timsort este folosit în Python, Java, Rust și multe alte limbaje populare, iar în scenarii uzuale poate depăși performanța `std::sort`, în special pe șiruri aproape sortate sau cu multe duplicate.

3.6. Sleep Sort

Această secțiune va fi cea mai detaliată, deoarece toți ceilalți algoritmi au fost discutați de mii de ori în literatura de specialitate până când a *adoarme* fiecare student la algoritmi și structuri de date, iar Sleep Sort nu are literatură de specialitate. Nu doresc să reinventez roata algoritmilor de sortare, mă interesează ce se întâmplă când roata *adoarme*.

Sleep sort este un algoritm creat de o persoană necunoscută pe platforma 4chan [9][14][8] [13][12][11] în 2011. Postarea [8] de pe 4chan a atras multă atenție asupra sa, având sute de comentarii, dar pentru oricine dorește să *doarmă* bine noaptea nu e recomandat de citit mai mult de primele mesaje, considerând natura nemoderată a platformei.

Acest algoritm funcționează prin crearea unui thread [20] pentru fiecare element din șir, unde fiecare thread "*doarme*" [19] pentru o perioadă de timp proporțională cu valoarea elementului său. După ce se trezește, thread-ul tipărește valoarea sa, rezultând un șir sortat în ordine crescătoare.

Deși e o idee foarte simplă, această metodă de sortare deschide ușa unor întrebări interesante:

- Cum determinăm timpul de *somn* corect pentru fiecare thread?
- Ce se întâmplă când thread-urile se trezesc în același timp, cum ar fi în cazul valorilor duplicate?

- Cum am putea măsura complexitatea acestui algoritm? Cum diferă performanța în funcție de sistemul de operare și hardware-ul folosit?
- Cum se compară performanța Sleep Sort cu alți algoritmi bazați pe altceva decât comparații, cum ar fi Counting Sort sau Radix Sort, în special pe șiruri cu multe duplicate sau cu o gamă limitată de valori?

```
void sleepSort(std::vector<int>& v) {
    std::mutex mtx;
    std::vector<int> r = v;
    v.clear();

    std::unordered_map<int, int> freq;
    for (int e : r) {
        if (e < 0)
            throw std::range_error(
                "Sleep sort cannot handle negative numbers: " +
                std::to_string(e));
        freq[e]++;
    }

    std::vector<std::thread> ths;
    ths.reserve(freq.size());

    for (auto& [val, count] : freq) {
        ths.push_back(std::thread([val, count, &v, &mtx]() {
            std::this_thread::sleep_for(
                std::chrono::milliseconds((val + 1) * 100));
            std::lock_guard<std::mutex> lock(mtx);
            for (int i = 0; i < count; i++)
                v.push_back(val);
        }));
    }

    for (auto& t : ths)
        if (t.joinable()) t.join();
}
```

Code 6. Implementarea Sleep Sort în C++.

Pentru a răspunde la aceste întrebări, am implementat o versiune modificată a Sleep Sort în C++, care gestionează eficient valorile duplicate și minimizează riscul de supraîncărcare a procesorului. În loc să creeze un thread pentru fiecare element, am folosit un `std::unordered_map` [18] pentru a grupa elementele identice și a le procesa împreună, reducând astfel numărul total de thread-uri necesare. Această implementare face Sleep Sort mult mai eficient pe date duplicate, cum ar fi cele din distribuția plată, și permite o evaluare mai realistă a performanței sale.

3.6.1. Determinarea timpului de somn

Pentru a determina timpul de *somn* corect pentru fiecare thread, principalii factori sunt rezoluția cronometrului [17] și modul în care sistemul de operare gestionează scheduling-ul thread-urilor [10]. În general, Sleep Sort poate fi ineficient pe sisteme cu rezoluție scăzută a timer-ului sau cu scheduling preemptiv agresiv, deoarece thread-urile pot fi trezite mai devreme sau mai târziu decât timpul specificat, afectând ordinea de sortare.

Deoarece nu există o metodă standardizată de a determina timpul de *somn* corect [11], am folosit o abordare empirică, ajustând timpul în funcție de valorile din șir și de performanța observată în testele preliminare. De multe ori Sleep Sort nu sorta corect cu timpuri prea mici, iar timpuri prea mari îl fac inutil de lent, am căutat astfel un echilibru între corectitudine și performanță.

Prioritizând corectitudinea, ceea ce Sleep Sort ar prefera pentru că, aidoma unui student, îi permite să *doarmă* mai mult, am ales un timp de *somn* de 100 ms pe unitate de valoare, conform formulei:

$$t_{\text{somn}}(x) = (x + 1) \times 100 \text{ ms} \quad (1)$$

unde x este valoarea elementului. Termenul $+1$ garantează că niciun element nu doarme 0 ms, inclusiv elementele cu valoarea 0.

3.6.2. Problema elementelor duplicate

Surprinzător, problema elementelor duplicate nu există. Sleep Sort funcționează perfect chiar și cu valori duplicate, deoarece fiecare thread doarme pentru o perioadă de timp proporțională cu valoarea sa, iar thread-urile care corespund valorilor identice vor dormi pentru aceeași perioadă și se vor trezi în același timp. Desigur, acest lucru înseamnă că elementele duplicate vor avea fiecare un thread separat, ceea ce e foarte ineficient și ceea ce am rezolvat prin folosirea unui `std::unordered_map` pentru a grupa elementele identice și a le procesa împreună.

3.6.3. Complexitatea algoritmului și influența sistemului de operare și hardware-ului

Complexitatea Sleep Sort nu poate fi analizată prin metodele clasice bazate pe comparații [11], deoarece algoritmul nu compară niciodată două elemente între ele. În schimb, complexitatea depinde de două componente independente:

Prima componentă este crearea și gestionarea thread-urilor [10, 20]. În implementarea propusă, numărul de thread-uri create este egal cu numărul de valori unice din șir, deci $O(k)$ unde $k \leq n$ este numărul de elemente distincte. Pe distribuția plată cu 5 valori distincte, $k = 5$ indiferent de n , un avantaj semnificativ față de varianta clasică.

A doua componentă este timpul total de execuție, determinat de cel mai lung interval de somn [17]:

$$T_{\text{total}} = (\max(A) + 1) \times 100 \text{ ms} \quad (2)$$

unde $\max(A)$ este valoarea maximă din șirul A . Această componentă este independentă de n , sortarea unui milion de elemente cu valori între 1 și 100 durează același timp ca sortarea a zece elemente cu aceleași valori.

Complexitatea spațiu este determinată de trei componente: vectorul de rezultate v de dimensiune $O(n)$, `std::unordered_map` [18] care stochează frecvențele celor k valori unice în $O(k)$, și vectorul de thread-uri de dimensiune $O(k)$, fiecare thread ocupând un spațiu fix pe stivă [10]. Prin urmare:

$$S(n, k) = O(n) + O(k) + O(k) = O(n + k) \quad (3)$$

Desigur, aceste calcule nu iau în considerare overhead-ul suplimentar cauzat de scheduling-ul thread-urilor și de rezoluția timer-ului, care pot afecta performanța reală a algoritmului în funcție de sistemul de operare și hardware-ul folosit [10, 17]. Acești factori sunt fundamental imposibili de modelat teoretic, deoarece depind de implementarea specifică a sistemului de operare și de arhitectura hardware, precum și de încărcarea curentă a sistemului și de prioritățile thread-urilor.

3.6.4. Comparația cu Counting și Radix Sort

Ambii algoritmi, Counting Sort și Radix Sort, sunt algoritmi de sortare non-comparativi care pot avea performanțe superioare `std::sort` în anumite condiții, în special pe șiruri cu o gamă limitată de valori sau cu multe duplicate [4].

Fără a mai menține suspans, Counting Sort și Radix Sort sunt mai rapizi decât Sleep Sort, dar nu mereu sunt cei mai eficienți.

O problemă majoră a counting sort este că necesită un vector auxiliar de dimensiune proporțională cu gama de valori din șir [4]. Dacă această gamă este foarte mare, cum ar fi în cazul valorilor întregi între 1 și 10^6 , counting sort devine ineficient din punct de vedere al spațiului, ocupând $O(k)$ unde k este gama de valori, ceea ce poate fi prohibitiv pentru șiruri mari. Sleep Sort nu necesită un astfel de vector auxiliar și poate gestiona eficient șiruri cu o gamă largă de valori, deși performanța sa depinde de rezoluția timer-ului și de scheduling-ul thread-urilor.

Un alt factor este că Sleep Sort exploatează în mod natural paralelismul hardware [10], thread-urile rulează concurrent, nu secvențial, ceea ce îl diferențiază fundamental de orice algoritm clasic de sortare. Acest fapt oferă Sleep Sort un avantaj dacă hardware-ul disponibil are resurse abundente, deci, cu destule nuclee și o rezoluție a timer-ului suficient de fină, nu este imposibil ca Sleep Sort să depășească performanța algoritmilor menționați anterior.

3.7. Ipoteze și așteptări

Table 1 prezintă o sinteză a complexităților teoretice și proprietăților algoritmilor analizați în această lucrare până acum.

Table 1. Complexitatea și proprietățile algoritmilor de sortare

Algoritm	Best	Average	Worst	Spațiu	Stabil
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Da
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Da
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Da
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	Nu
<code>std::sort</code>	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(\log n)$	Nu
Sleep Sort	$O(k)$	$O(k)$	$O(k)$	$O(n + k)$	Nu

k = numărul de valori unice din șir. Complexitatea Sleep Sort exclude timpul de așteptare $O(\max(A))$, care depinde de valorile elementelor, nu de n .

Stabilitatea Sleep Sort depinde de implementare — varianta propusă folosește `std::unordered_map`, care nu garantează ordinea de iterare, deci nu este stabilă.

Pe baza acestor proprietăți teoretice, ne așteptăm ca Merge Sort și `std::sort` să domine pe șiruri mari și aleatoare, Quick Sort să se degradeze pe distribuțiile sortate, iar Sleep Sort să surprindă pe distribuția plată unde $k = 5$ indiferent de n sau pe distribuții cu valori mici. Aceste ipoteze sunt evaluate empiric în Secțiunea ??, unde vom vedea dacă Sleep Sort se trezește la înălțimea așteptărilor sau dacă rămâne adormit în fața concurenței.

4. CROSS-REFERENCE COMMANDS*

Rho-class provides shorthand commands for cross-referencing figures, tables, code listings, and equations. To customize the label format, edit the `\rhobabel.sty` file. Label translations for Spanish are handled automatically.

- `\figref{}` — Uses the label style defined by `\figlabel`.
- `\figsref{}` — Uses the label style defined by `\figslabel`.
- `\tabref{}` — Uses the label style defined by `\tablalabel`.
- `\coderef{}` — Uses the label style defined by `\captionlabelcode`.
- `\equref{}` — Uses the label style defined by `\eqlabel`.

These commands generate fully hyperlinked references, both the label and the number, ensuring navigation to the referenced element when clicked in the PDF.

5. FIGURES

Fig. 1 shows a 3D surface visualization of a bivariate function.

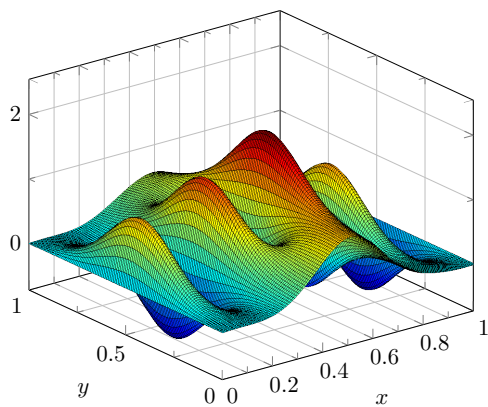


Figure 1. 3D surface obtained from PGFPlots [PFGPlots].

6. TABLES

Table 2 shows an example table of some astronomical objects.

Table 2. Astronomical Object Data

Object	Type	Distance (Light Years)
Alpha Centauri	Star system	4.37
Betelgeuse	Red supergiant star	642.5
Andromeda	Spiral galaxy	2.537 million
Earth	Planet	0
Sirius	Binary star system	8.6

Note: The table contains data of some famous celestial objects.

The `\tabletext{}` command is used to add notes to tables easily.

7. DOCUMENT DATA

7.1. Front-Matter Footer Block*

A new block called `\docinfo{}`, has been added to the bottom right of the first page, acting as a floating element which stays fixed in place. This replaces the corresponding author block previously located above the abstract in earlier versions.

The `\docinfo{}` block was created to give you greater control over supplementary document data, such as publication dates, licensing information, extended author details, and other front-matter notes. You can fully customize its content to suit your needs.

Additionally, the new `\corres{}` command serves as a flexible alternative to `\thanks{}`. Since modifying the standard `\thanks{}` command can be complex and its positioning is often rigid, `\corres{}` was introduced to offer a more adaptable solution without sacrificing functionality.

Note

You can use `\corres{}` as many times as needed since it automatically ensures that each entry remains correctly associated. For best results, I recommend using text symbols rather than mathematical ones (e.g., `\textasteriskcentered`, `\textdagger`, etc.).

If `\corres{}` is not used, `\docinfo{}` will automatically adjust its position. The implementation was designed to handle any combination of front-matter elements you might need when starting your document. In this example template, you'll find a minimal demonstration that combines both to illustrate their interaction.

7.2. Headers and Footers*

7.2.1. Headers

On all pages except the first, the document title and journal appear in the header. Since twoside layout is enabled, its position alternates depending on whether the page is odd or even.

7.2.2. Footers

In the first page, six elements display supplementary information for your document:

- `\journalname{}` — Displays the journal name, university, institute, company, or any institutional affiliation.
- `\theday{}` — Displays the current date.
- `\vol{}` — Specifies the volume number of the publication.
- `\no{}` — Specifies the issue number within the volume.
- `\journal{}` — Defines the full journal title.
- `\thepage` — Shows the current page number and the total count.

In all cases, you can freely rearrange or customize the order of these elements by modifying the class file to suit your needs. If any of them are not required, simply omit the corresponding command — the remaining elements will automatically adjust their spacing to remain evenly distributed.

8. RHO CUSTOM PACKAGES

8.1. Rhobabel.sty

This package have all the commands that automatically translate from English to Spanish when this custom package is defined.

By default, this document has its content in English. However, at the beginning of the document you will find a recommendation when writing in Spanish.

You may modify this package if you want to use other language than English or Spanish. This will make easier to translate your document without having to modify the class document.

8.2. Rhoenvs.sty

This package provides custom environments designed to enhance the visual presentation and structure of your document. Key examples include rhoenv, info, and note.

Their style is defined by `rhoenvstyle` — allowing you to customize their appearance according to your document's requirements.

An example using the `rhoenv` environment is shown below.

Custom Title

This is an example of the custom title environment. To add a title type this command `[frametitle=Custom Title]` next to the beginning of this environment (as shown in this example).

The `info` and `note` are environments which have a predefined title and translate their title to Spanish automatically when this language is defined.

9. EQUATIONS

Equation 4 shows the Schrödinger equation as an example.

$$i\hbar \frac{\partial}{\partial t} \Psi(\mathbf{r}, t) = \left[\frac{-\hbar^2}{2m} \nabla^2 + V(\mathbf{r}, t) \right] \Psi(\mathbf{r}, t)$$

(4)

The `amssymb` and `amsmath` packages were not required, as STIX2 font incorporates mathematical symbols for writing quality equations.

Note

If you would like to change the values that adjust the spacing above and below the equations, change the `\eqskip` value until the preferred spacing is set. The default value is set to 9pt.

10. CODES

10.1. Coding with Minted*

The `minted` package offers customized features for adding codes. In addition, the template is designed to work seamlessly with Fira Code, a monospaced font specially crafted for programming environments.

If you are using a desktop app as TeXstudio, try these steps to make this package work on Windows:

1. Install Python — A stable version (e.g., v.3.11)
2. Open the terminal and type `pip install Pygments`.
3. Update if a newer version is available.
4. Go to TeXStudio settings and change the default compiler to `pdflatex -shell-escape -interaction=nonstopmode \%.tex`.
5. Compile and wait for the result.

Caution

Ensure that `pygments` is properly installed and added to your system's PATH. Otherwise, you may encounter compilation errors. Additionally, enable shell escape when compiling, as it is required for minted to process and highlight code.

In Overleaf, this package is easier to use since it does not require any additional installations or modifications — just add the code as shown in the example. Code 7 shows a Python example.

```
# Fira Code Demo - Python
def main():
    # Common operators
    a = 10
    b = 20
    if a < b and a != 0:
        print("a is less than b")

    # Comparisons
    x = (a >= 5) and (b <= 30)

    # Walrus operator
    if (value := get_data()) is not None:
        process(value)

    # Lambda with arrow
    scale = lambda z: z * 2

    # Bitwise operations
    flag = a & b | (a ^ b)
    shift = a << 2 >> 1

    return flag, shift

def get_data():
    return "Simulated data"

def process(data):
    print(f"Processing: {data}")

if __name__ == "__main__":
    main()
```

Code 7. Python code example with minted.

You can customize its design changing `\usemintedstyle` command. The different styles offered by the `minted` package can be preview through this link — <https://pygments.org/styles/>.

10.2. Coding with Listings

Since `minted` requires additional installations and can be complex in some desktop TeX editors, you can use the `listings` package instead, which provides a simpler way to include code.

```
# Fira Code Demo - Python
def main():
```

```
# Common operators
a = 10
b = 20
if a < b and a != 0:
    print("a is less than b")

# Comparisons
x = (a >= 5) and (b <= 30)

# Walrus operator
if (value := get_data()) is not None:
    process(value)

# Lambda with arrow
scale = lambda z: z * 2

# Bitwise operations
flag = a & b | (a ^ b)
shift = a << 2 >> 1

return flag, shift

def get_data():
    return "Simulated data"

def process(data):
    print(f"Processing: {data}")

if __name__ == "__main__":
    main()
```

Code 8. Python code example with listings.

While `listings` is a simpler and widely supported package for code formatting, `minted` offers a more modern and powerful approach. It leverages the Pygments syntax highlighter to deliver superior coloring, language support, and styling options. One of its key advantages is the ability to easily switch between built-in color styles.

In contrast, `listings` requires manual setup for colors, fonts, and formatting rules. For users who prefer fine-tuned customization, the styling options for `listings` are organized in `rho.cls` file and can be modified at any time to suit individual preferences.

10.3. Inline Code*

As shown in this template, inline code has a custom style for improved visual appeal. To use it, simply add `\inlinecode{}` command and place your code inside the braces.

■ NUMBERLESS SECTION

Declaring a numberless section automatically inserts a square symbol before the title. This unique feature is specific to first-level sections within this class.

As this symbol also affects the table of contents and references titles, users who prefer to disable it may edit the section styling parameters in `rho.cls`.

11. BIBLIOGRAPHY

Bibliography management is handled by `biblatex`, with the default citation and reference style set to IEEE. The `citestyle=numeric-comp` option is enabled, allowing multiple citations to be grouped within a single bracket (e.g., [davis2023signalflow, bell2022codefont]). The citation format can be customized directly in `rho.cls` to suit any preferred style.

■ FAQ

How do I manage my references?

To manage your references, I recommend using the tool `scribbr`. You can simply enter the URL or create your own citation, and then export it to L^AT_EX using the options in the three-dot menu.

The generated citation can be copied and pasted into `rho.bib`, the file designated for bibliography management. You may rename this file, but if you do, remember to update the `\addbibresource` command in `rho.cls` under the biblatex section.

Note

Some platforms, such as Google Scholar or scientific journals, provide citations directly in BibTeX format. Therefore, check if there is a “how to cite this document” section to streamline the citation process even further.

If you have any further questions, you can refer to the following page — Bibliography management with biblatex.

What should I do with the example files?

The template includes sample content — such as an example figure, bibliography entries, and the `example.py` script — to demonstrate how the layout and features work. Once you start customizing the document for your own use, feel free to delete all example files and entries you don’t need.

How do I place equations easily?

For equations, we have two options: inline or on its own line. For inline equations, simply place a dollar sign (\$) at the beginning and end of the equation. However, if you want the equation to be displayed on its own line, you need to use the equation environment.

If you find it challenging to write formulas directly in $\text{\LaTeX}$, you can use text editors like Word. In the equations menu, you can select $\text{\LaTeX}$ in the conversion section and copy and paste the equation you wrote into one of these two environments.

How do I change the paper size?

By default, this class was adapted for a4paper and test it with letterpaper. The following paper sizes are available in $\text{\LaTeX}$:

- letterpaper (11 × 8.5 in)
- legalpaper (14 × 8.5 in)
- executivepaper (10.5 × 7.25 in)
- a4paper (21 × 29.7 cm)
- a5paper (21 × 14.8 cm)
- b5paper (25 × 17.6 cm)

CONTACT ME

Have questions, suggestions, or an idea for a new feature? Found a bug or working on a project you’d like to invite me to?

Feel free to reach out — I’d be happy to help, collaborate, or fix the issue. Your feedback helps me improve my templates!

✉ memo.notess1@gmail.com
🌐 memonotess.com
📷 @memo.notess

GITHUB REPOSITORY

Visit the repository to access the source code, track ongoing development, report issues, and stay up to date with the latest changes.

🔗 <https://github.com/MemoJimenez/Rho-class>

SUPPORTING

Enjoyed this document class? Check out tau-class, my very first $\text{\LaTeX}$ template.

🔗 <https://es.overleaf.com/latex/templates/tau-class>

Any contributions are welcome!

Coffee keeps me awake and helps me create better $\text{\LaTeX}$ templates. If you wish to support my work, you can do so through PayPal:

👉 <https://www.paypal.me/GuillermoJimenez>

Enjoy writing with rho-class!

REFERENCES

- [1] C. A. R. Hoare, “Algorithm 64: Quicksort”, *Communications of the ACM*, vol. 4, no. 7, p. 321, 1961. DOI: 10.1145/366622.366644
- [2] E. W. Dijkstra, *A Discipline of Programming*. Prentice Hall, 1976.
- [3] D. R. Musser, “Introspective sorting and selection algorithms”, *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997. DOI: 10.1002/(SICI)1097-024X(199708)27:8<983::AID-SPE117>3.0.CO;2-#
- [4] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998.
- [5] T. Peters, “Timsort”, Python Software Foundation, Tech. Rep., 2002.
- [6] O. Astrachan, “Bubble sort: An archaeological algorithmic analysis”, in *Proceedings of the 34th SIGCSE Technical Symposium*, ACM, 2003, pp. 1–5.
- [7] M. Bender, M. Farach-Colton, and M. Mosteiro, “Insertion sort is $O(n \log n)$ ”, *Theory of Computing Systems*, vol. 39, pp. 391–397, 2006. DOI: 10.1007/s00224-005-1237-z
- [8] Anonymous, *Genius sorting algorithm: Sleep sort*, 4chan /prog/, arhivat la <https://archive.fo/xhGo>, 2011.
- [9] R. Sedgewick and K. Wayne, *Algorithms, 4th edition — lecture slides: Tries*, Princeton University COS226, <https://www.cs.princeton.edu/courses/archive/fall13/cos226/lectures/52Tries.pdf>, 2013.
- [10] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Pearson, 2014.
- [11] S. Rao, *Sleep sort: Where theory meets sobering reality*, DEV Community, <https://dev.to/sishaarrao/sleep-sort-where-theory-meets-sobering-reality-b3m>, 2017.
- [12] OpenGenus Foundation, *Sleep sort*, <https://iq.opengenus.org/sleep-sort/>, 2019.
- [13] GeeksforGeeks, *Sleep sort – the king of laziness*, <https://www.geeksforgeeks.org/sleep-sort-king-laziness-sorting-sleeping/>, 2023.
- [14] K. Henney, “Need something sorted? sleep on it!”, *ACCU Overload*, vol. 31, no. 175, 2023.
- [15] cppreference.com, *Std::sort*, <https://en.cppreference.com/w/cpp/algorithm/sort>, Accesat: 2024, 2024.
- [16] GCC libstdc++ contributors, *Stl_algo.h — GNU C++ standard library source*, https://github.com/gcc-mirror/gcc/blob/master/libstdc++-v3/include/bits/stl_algo.h, Accesat: 2024, 2024.
- [17] Microsoft, *Timer resolution*, <https://learn.microsoft.com/en-us/windows/win32/winmsg/wm-timer>, 2024.
- [18] *Std::map*, <https://en.cppreference.com/w/cpp/container/map>, 2024.
- [19] *Std::this_thread::sleep_for*, https://en.cppreference.com/w/cpp/thread/sleep_for, 2024.
- [20] *Std::thread*, <https://en.cppreference.com/w/cpp/thread/thread>, 2024.